

Apache Kafka

Complete In-Depth Guide – From Scratch

Architecture • Brokers • Topics • Partitions • Replication

Producers • Consumers • Consumer Groups • Offsets

Kafka Streams • Kafka Connect • Schema Registry • ksqlDB

Security • Performance Tuning • Operations • Spring Boot Integration

Clear Definitions • Architecture Diagrams • Full Code Examples • CLI Commands

Table of Contents

1. Kafka Fundamentals & Architecture

- 1.1 What is Apache Kafka?
- 1.2 Kafka vs Traditional Messaging
- 1.3 Core Architecture – Brokers, ZooKeeper & KRaft
- 1.4 Topics, Partitions & Offsets
- 1.5 Replication – Leader, Follower & ISR
- 1.6 Log Segments & Retention
- 1.7 Installing & Starting Kafka

2. Kafka Producers

- 2.1 Producer Architecture & Internal Flow
- 2.2 Producer Configuration Deep Dive
- 2.3 Acknowledgements – acks=0, 1, all
- 2.4 Partitioning Strategies
- 2.5 Idempotent & Transactional Producers
- 2.6 Producer with Java API – Full Example
- 2.7 Producer Performance Tuning

3. Kafka Consumers

- 3.1 Consumer Architecture & Poll Loop
- 3.2 Consumer Groups & Partition Assignment
- 3.3 Offset Management – Auto vs Manual Commit
- 3.4 Rebalancing – Strategies & Protocols
- 3.5 Consumer Configuration Deep Dive
- 3.6 Consumer with Java API – Full Example
- 3.7 Exactly-Once, At-Least-Once, At-Most-Once

4. Kafka Administration & CLI

- 4.1 Topic Management CLI
- 4.2 Consumer Group CLI
- 4.3 Producer & Consumer Console Tools
- 4.4 Partition Reassignment
- 4.5 Broker Configuration Deep Dive
- 4.6 Monitoring – JMX Metrics & Key KPIs

5. Kafka Streams

- 5.1 What is Kafka Streams?
- 5.2 Stream Processing Topology
- 5.3 KStream vs KTable vs GlobalKTable
- 5.4 Stateless Operations
- 5.5 Stateful Operations – Count, Reduce, Aggregate
- 5.6 Windowing – Tumbling, Hopping, Session
- 5.7 Stream-Table Joins
- 5.8 Interactive Queries

6. Kafka Connect

- 6.1 What is Kafka Connect?

- 6.2 Source & Sink Connectors
- 6.3 Connector Configuration
- 6.4 JDBC Source Connector
- 6.5 Elasticsearch Sink Connector
- 6.6 Debezium CDC Connector
- 6.7 REST API Management

7. Schema Registry & Avro

- 7.1 Why Schema Management?
- 7.2 Confluent Schema Registry
- 7.3 Apache Avro Format
- 7.4 Schema Evolution & Compatibility
- 7.5 Producing & Consuming Avro with Java
- 7.6 ksqldb Overview

8. Kafka Security

- 8.1 Encryption – TLS/SSL
- 8.2 Authentication – SASL Mechanisms
- 8.3 Authorisation – ACLs
- 8.4 Mutual TLS (mTLS)
- 8.5 Security Best Practices

9. Spring Boot + Kafka

- 9.1 Spring Kafka Setup
- 9.2 KafkaTemplate – Producer
- 9.3 @KafkaListener – Consumer
- 9.4 Error Handling & DLQ
- 9.5 Transactions with @Transactional
- 9.6 Testing Kafka with EmbeddedKafka

10. Performance Tuning & Best Practices

- 10.1 Producer Tuning
- 10.2 Consumer Tuning
- 10.3 Broker Tuning
- 10.4 Topic Design Best Practices
- 10.5 Monitoring & Alerting
- 10.6 Common Pitfalls & Troubleshooting

CHAPTER 1

Kafka Fundamentals & Architecture

Event Streaming Platform – Brokers, Topics, Partitions, Replication & Storage

1.1 What is Apache Kafka?

Apache Kafka

Apache Kafka is a distributed, fault-tolerant, high-throughput event streaming platform originally developed at LinkedIn and open-sourced in 2011. Kafka can publish, subscribe to, store, and process streams of records (events) in a fault-tolerant, durable manner. It is used for building real-time data pipelines, event-driven architectures, stream processing applications, and as a central nervous system connecting microservices.

- **High Throughput** – Handles millions of messages per second with very low latency.
- **Durability** – Messages are persisted to disk and replicated across brokers.
- **Scalability** – Horizontally scalable by adding brokers and partitions.
- **Fault Tolerance** – Automatic leader election and replication handle broker failures.
- **Replayability** – Consumers can re-read messages from any offset at any time.
- **Decoupling** – Producers and consumers are completely independent.

Feature	Traditional MQ (RabbitMQ)	Apache Kafka
Message Model	Queue or Pub/Sub	Distributed commit log
Message Deletion	Deleted after consumption	Retained by configurable TTL
Replay	Not possible once consumed	Any consumer can re-read from any offset
Throughput	Moderate (tens of thousands/sec)	Millions of messages per second
Consumers	Message disappears after one ack	Multiple independent consumer groups
Order	FIFO within queue	Strict order within a partition
Storage	In-memory + brief disk	Persistent disk storage (days/months)
Scalability	Vertical + limited horizontal	Horizontal: add brokers/partitions
Use Case	Task queues, RPC, work distribution	Event streaming, CDC, audit log, analytics

1.2 Core Architecture

Key Architectural Components:

Component	Role
Broker	A Kafka server node. Receives and stores messages. A cluster has 3+ brokers.
Cluster	A group of brokers working together; managed by ZooKeeper or KRaft.
ZooKeeper	Distributed coordinator (pre-Kafka 3.3): stores cluster metadata, leader elections.
KRaft Mode	Kafka 3.3+ replaces ZooKeeper with built-in Raft consensus protocol.
Topic	A named category/feed of records. Like a database table for events.
Partition	A topic is split into N ordered, immutable partitions for parallelism.
Offset	Sequential integer ID for each record within a partition. Starts at 0.
Producer	Client that publishes records to topics.
Consumer	Client that subscribes to topics and reads records.
Consumer Group	Logical grouping of consumers that share the work of reading a topic.
Replica	Copy of a partition stored on a different broker for fault tolerance.
Leader Replica	The partition replica that handles all reads and writes.
Follower Replica	Passive copies that replicate from the leader. Become leader on failure.
ISR	In-Sync Replicas: follower replicas that are caught up with the leader.
Record / Event	The fundamental unit of data: key, value, timestamp, headers.

1.3 Topics, Partitions & Offsets

Topic

A topic is a logical name for a stream of records. Topics are multi-subscriber: many producers can write to a topic, and many consumer groups can independently read from it. Topics are divided into partitions for scalability.

Partition

Each partition is an ordered, immutable sequence of records stored as a log file on disk. Records within a partition are assigned sequential offsets starting from 0. Partitions enable parallelism: multiple consumers in a group each own some partitions.

Offset

An offset is a unique, sequential identifier for each record within a partition. Consumers track their position by committing offsets. You can reset to any offset to replay messages.

Topic/Partition/Consumer model

```
# Topic with 3 partitions, replication factor 3

Topic: order-events
Partition 0: [0][1][2][3][4][5] <- records with offsets 0-5
Partition 1: [0][1][2][3] <- records with offsets 0-3
Partition 2: [0][1][2][3][4] <- records with offsets 0-4
Consumer Group A: 3 consumers, each owns 1 partition
```

```

Consumer A1 <- Partition 0 (at offset 5)
Consumer A2 <- Partition 1 (at offset 3)
Consumer A3 <- Partition 2 (at offset 4)

Consumer Group B: 1 consumer, reads ALL partitions
Consumer B1 <- Partition 0 + 1 + 2

Producer: sends to partition based on key hash or round-robin
Key="ORDER-1" -> hash -> Partition 0
Key="ORDER-2" -> hash -> Partition 2
Key=null -> Round-robin (0, 1, 2, 0, 1...)

```

1.4 Replication – Leader, ISR & Fault Tolerance

Replication Factor

The replication factor determines how many copies of each partition exist across brokers. A factor of 3 means 1 leader + 2 followers. The leader handles all reads and writes; followers replicate asynchronously. If the leader fails, one of the ISR followers is elected as the new leader in seconds.

Replication topology

```

# Replication with 3 brokers, replication factor 3

Topic: payments | Partition 0
Broker 1: [LEADER] <- all reads and writes go here
Broker 2: [FOLLOWER] <- replicating from leader (ISR)
Broker 3: [FOLLOWER] <- replicating from leader (ISR)

# If Broker 1 goes down:
ZooKeeper/KRaft detects failure in ~3 seconds
Elects Broker 2 or 3 as new LEADER
Producers and consumers automatically reconnect
Zero data loss (messages were replicated before ack)

# min.insync.replicas = 2 (recommended)
# Producer can only write if at least 2 replicas are in sync
# Prevents data loss even if a replica is slightly behind

```

1.5 Installing & Starting Kafka

Installing and starting Kafka – KRaft, ZooKeeper & Docker

```

# Download Kafka
wget https://downloads.apache.org/kafka/3.6.0/kafka_2.13-3.6.0.tgz
tar -xzf kafka_2.13-3.6.0.tgz && cd kafka_2.13-3.6.0

# ■■■ Option A: KRaft Mode (no ZooKeeper, recommended for 3.3+) ■■■

# 1. Generate a cluster UUID
KAFKA_CLUSTER_ID=$(bin/kafka-storage.sh random-uuid)

# 2. Format the storage directory
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties

# 3. Start the Kafka broker (acts as both broker and controller)
bin/kafka-server-start.sh config/kraft/server.properties

```

```
# ■■■ Option B: ZooKeeper Mode (legacy) ████████████████████████████████  
# 1. Start ZooKeeper first  
bin/zookeeper-server-start.sh config/zookeeper.properties  
  
# 2. Start Kafka broker  
bin/kafka-server-start.sh config/server.properties  
  
# ■■ Docker Compose (quickest for development) ████████████████████████████████  
# docker-compose.yml  
services:  
kafka:  
image: confluentinc/cp-kafka:7.5.0  
ports: ["9092:9092","9093:9093"]  
environment:  
KAFKA_NODE_ID: 1  
KAFKA_PROCESS_ROLES: broker,controller  
KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093  
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092  
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093  
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1  
CLUSTER_ID: MkU3OEVBNtcwNTJENDM2Qg  
  
kafka-ui:  
image: provectuslabs/kafka-ui:latest  
ports: ["8090:8080"]  
environment:  
KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
```

CHAPTER 2

Kafka Producers

Producer Architecture, Configuration, Acknowledgements, Partitioning & Transactions

2.1 Producer Internal Flow

- **1. Serialisation** – Key and value are serialised to bytes using configured Serializers.
- **2. Partitioning** – Partitioner selects which partition to send to (hash of key, or round-robin).
- **3. RecordAccumulator** – Records are batched in an in-memory buffer (batch.size bytes).
- **4. Sender Thread** – Background thread drains batches from accumulator and sends to brokers.
- **5. NetworkClient** – Manages TCP connections to broker leaders; handles retries.
- **6. Acknowledgement** – Broker sends ack back; Sender marks the batch as complete.

2.2 Producer Configuration Deep Dive

Property	Default	Description
bootstrap.servers	required	Comma-separated list of broker host:port pairs
key.serializer	required	Serializer class for record keys (StringSerializer, etc.)
value.serializer	required	Serializer class for record values
acks	all	Number of broker replicas that must ack a write (0, 1, all/-1)
retries	2147483647	Max number of retries on transient failures
retry.backoff.ms	100	Wait time between retry attempts
linger.ms	0	Wait up to N ms to fill a batch before sending
batch.size	16384 (16KB)	Max bytes in one batch per partition
buffer.memory	33554432 (32MB)	Total memory for the producer buffer
compression.type	none	Compress batches: none, gzip, snappy, lz4, zstd
max.in.flight.requests.per.connection	5	Max unacked requests; set to 1 for strict ordering
enable.idempotence	true	Exactly-once per-partition delivery; auto-sets acks=all

[illegible]

2.5 Transactional Producer

Transactional Producer – atomic multi-topic writes

```
// Transactional producer - atomically send to multiple topics
Properties props = new Properties();
// ... standard config ...
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "order-service-txn-1");
```

```
// transactional.id must be unique per producer instance

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
producer.initTransactions(); // must call once before any transactions

try {
    producer.beginTransaction();

    // Send to multiple topics atomically
    producer.send(new ProducerRecord<>("order-events", orderId, orderJson));
    producer.send(new ProducerRecord<>("audit-log", orderId, auditJson));
    producer.send(new ProducerRecord<>("inventory-events", itemId, stockJson));

    producer.commitTransaction(); // all three are visible atomically
} catch (Exception e) {
    producer.abortTransaction(); // none of them are visible
    throw e;
}
```

CHAPTER 3

Kafka Consumers

Consumer Groups, Offset Management, Rebalancing & Delivery Semantics

3.1 Consumer Groups & Partition Assignment

Consumer Group

A consumer group is a logical group of consumers that cooperate to consume a topic. Each partition is assigned to exactly ONE consumer within the group at any time. This provides parallel consumption: with N partitions and N consumers in a group, each consumer reads one partition. Extra consumers beyond the partition count are idle.

Consumer group partition assignment

```
# Rule: partitions are distributed across consumers in a group

Topic: payments (6 partitions: P0 P1 P2 P3 P4 P5)

Consumer Group: payment-processors (3 consumers)
Consumer 1: P0, P1 <- each consumer owns 2 partitions
Consumer 2: P2, P3
Consumer 3: P4, P5

Consumer Group: audit-service (1 consumer)
Consumer 1: P0,P1,P2,P3,P4,P5 <- single consumer reads ALL partitions

Consumer Group: analytics (7 consumers)
Consumer 1: P0
Consumer 2: P1
... (6 consumers each own 1 partition)
Consumer 7: IDLE <- extra consumer, no partition assigned!

# Key rule: # partitions = max useful consumers in a group
```

3.2 Offset Management

Offset Commit

The consumer tracks its read position by committing offsets to a special internal Kafka topic: `__consumer_offsets`. Committing offset N means 'I have successfully processed all messages up to and including offset N-1'. On restart, the consumer resumes from the last committed offset.

Commit Mode	Property	Behaviour	Risk
Auto Commit	<code>enable.auto.commit=true</code>	Commits every <code>auto.commit.interval.ms</code>	At-least-once or at-most-once depending on timing

Commit Mode	Property	Behaviour	Risk
Manual Sync	enable.auto.commit=false + commitSync()	Commits after processing; blocks until ack	At-least-once; slower
Manual Async	enable.auto.commit=false + commitAsync()	Commits without blocking; callback on result	At-least-once; no retry on failure
Specific Offset	commitSync(offsets)	Commits a specific partition/offset map	Fine-grained control

3.3 Full Java Consumer Example

Full Java Consumer with manual commit and rebalance listener

```
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;

public class OrderEventConsumer {

    public static void main(String[] args) {

        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "order-processors");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            StringDeserializer.class.getName());

        // Offset reset: "earliest" | "latest" | "none"
        // earliest = start from beginning if no committed offset exists
        // latest = start from the end (new messages only)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

        // Manual offset commit for at-least-once semantics
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);

        // Performance tuning
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);
        props.put(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, 1);
        props.put(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, 500);

        try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props)) {

            // Subscribe to one or more topics
            consumer.subscribe(List.of("order-events", "payment-events"));

            // Register rebalance listener
            consumer.subscribe(List.of("order-events"), new ConsumerRebalanceListener() {
                @Override
                public void onPartitionsRevoked(Collection<TopicPartition> partitions) {
                    // Commit offsets before partition is taken away
                    consumer.commitSync();
                }
            });
        }
    }
}
```

[illegible]

3.4 Delivery Semantics

Semantics	How Achieved	Risk	Use Case
At Most Once	Commit offset BEFORE processing	Message loss if processing fails	Logs, metrics where loss is OK
At Least Once	Commit offset AFTER processing	Duplicate processing on consumer restart	Most microservices – make processing idempotent
Exactly Once	Transactions + idempotent producers + read_committed isolation	Complex setup	Financial transactions, inventory

Kafka Administration & CLI

4.1 Topic Management CLI

[illegible]

```
# Type: ORDER-1:{ "status": "PLACED", "amount": 99.99}

# ■■ Consume Messages ████████████████████████████████████████████████████████████

bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic order-events \
--from-beginning \
--property print.key=true \
--property print.timestamp=true \
--group my-test-group

# ■■ Consumer Group Management ████████████████████████████████████████████████████████████

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--describe --group order-processors

# Shows: LAG per partition (how far behind the consumer is)

# Reset consumer offset to beginning (replay all messages)

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--group order-processors \
--topic order-events \
--reset-offsets --to-earliest \
--execute

# Reset to specific offset

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
--group order-processors \
--topic order-events:2 \ # partition 2 only
--reset-offsets --to-offset 500 \
--execute
```


Kafka Streams

5.1 What is Kafka Streams?

Kafka Streams is a client library embedded in your application for building real-time streaming applications using Kafka as both input and output. Unlike Spark or Flink, it requires NO separate processing cluster – it runs inside your JVM process. It supports stateless and stateful operations, time-based windowing, stream-table joins, and fault-tolerant local state stores.

5.2 Stateless Operations

```
Properties props = new Properties();
props.put(StreamsConfig.APPLICATION_ID_CONFIG, "order-stream-app");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
Serdes.String().getClass());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
Serdes.String().getClass());

StreamsBuilder builder = new StreamsBuilder();

// Source Stream
KStream<String, String> orders =
builder.stream("order-events");

// filter - keep only CONFIRMED orders
KStream<String, String> confirmed =
```

[illegible]

5.3 Stateful Operations & Aggregation

Stateful aggregation – count, aggregate, reduce

[illegible]

[illegible]

5.4 Windowing

Window Type	Description	Example
Tumbling	Fixed size, non-overlapping; every event belongs to exactly one window	Revenue per 1-hour bucket: [0-1h][1-2h][2-3h]
Hopping	Fixed size, overlapping; events can appear in multiple windows	Moving average: 1h window, 15min hop
Session	Dynamic size based on activity gaps; no activity = window closes	User session: close after 30min inactivity
Sliding	Event-time windows that slide on every new event	All events within 10 minutes of each other

Tumbling and Session Windows

```
// Tumbling Window - revenue per 1-hour bucket
KTable<Windowed<String>, Double> hourlyRevenue =
    orderStream
        .groupBy((k, v) -> v.getCategory(),
            Grouped.with(Serdes.String(), orderEventSerde))
        .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(1)))
        .aggregate(
            () -> 0.0,
            (key, order, total) -> total + order.getAmount(),
            Materialized.<String, Double, WindowStore<Bytes, byte[]>>
                as("hourly-revenue").withValueSerde(Serdes.Double())
        );

// Convert windowed result to stream with window time in key
```

```
hourlyRevenue.toStream()
    .map((windowedKey, revenue) -> KeyValue.pair(
        windowedKey.key() + "@" + windowedKey.window().startTime(),
        revenue))
    .to("hourly-revenue");

// Session Window - user activity sessions
KTable<Windowed<String>, Long> userSessions =
    clickStream
        .groupByKey()
        .windowedBy(SessionWindows.ofInactivityGapWithNoGrace(Duration.ofMinutes(30)))
        .count(Materialized.as("session-store"));
```

CHAPTER 6

Kafka Connect

Source & Sink Connectors, JDBC, Elasticsearch, Debezium CDC & REST API

6.1 What is Kafka Connect?

Kafka Connect

Kafka Connect is a framework for reliably streaming data between Kafka and external systems (databases, file systems, search engines, cloud services). Instead of writing custom producer/consumer code, you configure a Connector – a plugin that handles the integration. It provides automatic offset management, error handling, and scalability.

Connector Type	Direction	Examples
Source Connector	External System → Kafka	JDBC Source (DB → Kafka), Debezium CDC, File Source, Twitter
Sink Connector	Kafka → External System	Elasticsearch Sink, S3 Sink, JDBC Sink, HDFS Sink

6.2 JDBC Source Connector – Database to Kafka

JDBC Source Connector config

```
# JDBC Source Connector configuration (POST to /connectors)
{
  "name": "jdbc-orders-source",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "connection.url": "jdbc:postgresql://localhost:5432/ecommerce",
    "connection.user": "kafka_user",
    "connection.password": "password",
    "table.whitelist": "orders,order_items",
    "mode": "timestamp+incrementing", // detect new and updated rows
    "timestamp.column.name": "updated_at",
    "incrementing.column.name": "id",
    "topic.prefix": "db.", // creates topics db.orders, db.order_items
    "poll.interval.ms": "5000", // poll DB every 5 seconds
    "transforms": "copyFieldToKey",
    "transforms.copyFieldToKey.type":
      "org.apache.kafka.connect.transforms.ValueToKey",
    "transforms.copyFieldToKey.fields": "id",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter"
```

```
}  
}
```

Debezium CDC Connector – capture inserts, updates, deletes

```
# Debezium PostgreSQL CDC Connector – capture every DB change  
{  
  "name": "debezium-orders-cdc",  
  "config": {  
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",  
    "database.hostname": "localhost",  
    "database.port": "5432",  
    "database.user": "debezium",  
    "database.password": "dbz",  
    "database.dbname": "ecommerce",  
    "topic.prefix": "cdc",  
    "table.include.list": "public.orders,public.payments",  
    "plugin.name": "pgoutput",  
    "publication.autocreate.mode": "filtered",  
    "key.converter": "io.confluent.connect.avro.AvroConverter",  
    "value.converter": "io.confluent.connect.avro.AvroConverter",  
    "key.converter.schema.registry.url": "http://schema-registry:8081",  
    "value.converter.schema.registry.url": "http://schema-registry:8081"  
  }  
}
```

Each DB operation becomes an event:

INSERT -> {before: null, after: {id:1, amount:99.99, ...}, op: "c"}

UPDATE -> {before: {old_values}, after: {new_values}, op: "u"}

DELETE -> {before: {deleted_row}, after: null, op: "d"}

Kafka Connect REST API

```
# Connect REST API management  
  
# List all connectors  
curl http://localhost:8083/connectors  
  
# Create a connector  
curl -X POST -H "Content-Type: application/json" \  
http://localhost:8083/connectors \  
-d @connector-config.json  
  
# Get connector status  
curl http://localhost:8083/connectors/jdbc-orders-source/status  
  
# Pause / Resume  
curl -X PUT http://localhost:8083/connectors/jdbc-orders-source/pause  
curl -X PUT http://localhost:8083/connectors/jdbc-orders-source/resume  
  
# Restart a failed connector  
curl -X POST http://localhost:8083/connectors/jdbc-orders-source/restart  
  
# Delete a connector
```

```
curl -X DELETE http://localhost:8083/connectors/jdbc-orders-source
```

CHAPTER 7

Schema Registry & Avro

Schema Evolution, Compatibility, Avro Serialisation & ksqlDB

7.1 Why Schema Management?

Schema Registry

As systems evolve, the structure of events changes. Without a schema registry, producers can send events with new fields that break existing consumers (or remove fields consumers depend on). The Confluent Schema Registry provides a centralised repository for schemas, enforces compatibility rules, and stores schema versions. Avro, Protobuf, and JSON Schema are all supported.

Compatibility Type	Rule	Use When
BACKWARD	New schema can read data written with old schema	Consumer updates before producer
FORWARD	Old schema can read data written with new schema	Producer updates before consumer
FULL	Both backward AND forward compatible	Unknown rollout order
NONE	No compatibility checks enforced	Development; never in production

- **BACKWARD-compatible changes:** add optional fields (with default), remove optional fields.
- **FORWARD-compatible changes:** remove fields, add fields with default.
- **BREAKING changes:** rename a field, change a field type (int to string), remove required field.

7.2 Avro Schema & Java Integration

Avro schema definition

```
// Order event Avro schema: src/main/avro/OrderEvent.avsc
{
  "type": "record",
  "name": "OrderEvent",
  "namespace": "com.example.events",
  "fields": [
    {"name": "orderId", "type": "string"},
    {"name": "customerId", "type": "string"},
    {"name": "amount", "type": "double"},
    {"name": "status", "type": {
      "type": "enum",
      "name": "OrderStatus",
      "symbols": ["PLACED", "CONFIRMED", "SHIPPED", "CANCELLED"]
    }}
  ]
}
```



```

    }},
    {"name": "items", "type": {
      "type": "array",
      "items": {
        "type": "record",
        "name": "OrderItem",
        "fields": [
          {"name": "productId", "type": "string"},
          {"name": "quantity", "type": "int"},
          {"name": "price", "type": "double"}
        ]
      }
    }},
    {"name": "notes", "type": ["null", "string"], "default": null}
  ]
}

```

Avro Producer & Consumer with Schema Registry

```

<!-- pom.xml - Avro + Schema Registry -->
<dependency>
  <groupId>io.confluent</groupId>
  <artifactId>kafka-avro-serializer</artifactId>
  <version>7.5.0</version>
</dependency>

<dependency>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro</artifactId>
  <version>1.11.3</version>
</dependency>

// Producer with Avro + Schema Registry
Properties props = new Properties();
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
KafkaAvroSerializer.class);
props.put("schema.registry.url", "http://localhost:8081");
props.put("auto.register.schemas", true);

try (KafkaProducer<String, OrderEvent> producer = new KafkaProducer<>(props)) {
  OrderEvent event = OrderEvent.newBuilder()
    .setOrderId("ORDER-123")
    .setCustomerId("CUST-456")
    .setAmount(99.99)
    .setStatus(OrderStatus.PLACED)
    .setItems(List.of(
      new OrderItem("PROD-1", 2, 49.99)))
    .build();
}

```

```
producer.send(new ProducerRecord<>("order-events", event.getOrderId(), event));
}

// Consumer with Avro
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
KafkaAvroDeserializer.class);
props.put("specific.avro.reader", true); // use generated class, not GenericRecord

try (KafkaConsumer<String, OrderEvent> consumer = new KafkaConsumer<>(props)) {
    consumer.subscribe(List.of("order-events"));
    ConsumerRecords<String, OrderEvent> records = consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String, OrderEvent> record : records) {
        OrderEvent order = record.value();
        System.out.println("Order: " + order.getOrderId() +
            " Amount: " + order.getAmount());
    }
}
```

CHAPTER 8

Kafka Security

TLS Encryption, SASL Authentication, ACL Authorisation & Best Practices

8.1 Security Layers

Security Layer	Mechanism	Protects Against
Encryption in Transit	TLS/SSL	Network eavesdropping; man-in-the-middle
Authentication	SASL/PLAIN, SCRAM, Kerberos	Unauthorised clients connecting
Authorisation	ACLs (Access Control Lists)	Authenticated clients reading/writing wrong topics
Encryption at Rest	OS/Volume encryption	Physical disk theft

8.2 TLS Configuration

TLS configuration for broker and client

```
# server.properties - broker TLS config
listeners=SSL://0.0.0.0:9093
ssl.keystore.location=/opt/kafka/certs/kafka.server.keystore.jks
ssl.keystore.password=serverpassword
ssl.key.password=serverpassword
ssl.truststore.location=/opt/kafka/certs/kafka.server.truststore.jks
ssl.truststore.password=serverpassword
ssl.client.auth=required # mTLS: clients must present certificate

# Client (producer/consumer) SSL config
security.protocol=SSL
ssl.truststore.location=/opt/kafka/certs/client.truststore.jks
ssl.truststore.password=clientpassword
ssl.keystore.location=/opt/kafka/certs/client.keystore.jks
ssl.keystore.password=clientpassword
```

8.3 SASL Authentication & ACLs

SASL/SCRAM authentication and ACL management

```
# SASL/SCRAM-SHA-512 broker config
listeners=SASL_SSL://0.0.0.0:9094
security.inter.broker.protocol=SASL_SSL
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512
```

```
sasl.enabled.mechanisms=SCRAM-SHA-512

# Create user credentials (SCRAM)
bin/kafka-configs.sh --bootstrap-server localhost:9092 \
--alter --entity-type users --entity-name alice \
--add-config 'SCRAM-SHA-512=[password=alice-secret]'

# ACL Examples

# Allow alice to produce to order-events
bin/kafka-acls.sh --bootstrap-server localhost:9092 \
--add --allow-principal User:alice \
--producer --topic order-events

# Allow bob to consume from order-events (consumer group)
bin/kafka-acls.sh --bootstrap-server localhost:9092 \
--add --allow-principal User:bob \
--consumer --topic order-events \
--group order-processors

# Allow service account to read/write ALL topics with prefix "app."
bin/kafka-acls.sh --bootstrap-server localhost:9092 \
--add --allow-principal User:service-account \
--operation READ --operation WRITE \
--topic app. --resource-pattern-type PREFIX

# List ACLs
bin/kafka-acls.sh --bootstrap-server localhost:9092 --list
```

CHAPTER 9

Spring Boot + Apache Kafka

KafkaTemplate, @KafkaListener, Error Handling, DLQ, Transactions & Testing

9.1 Setup & Configuration

Spring Kafka configuration

```
<!-- pom.xml -->
<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>

# application.yml
spring:
kafka:
bootstrap-servers: localhost:9092
producer:
key-serializer: org.apache.kafka.common.serialization.StringSerializer
value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
acks: all
retries: 3
properties:
enable.idempotence: true
linger.ms: 20
batch.size: 32768
compression.type: snappy
consumer:
group-id: order-service
auto-offset-reset: earliest
enable-auto-commit: false
key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
properties:
spring.json.trusted.packages: "com.example.events"
listener:
ack-mode: MANUAL_IMMEDIATE
concurrency: 3 # 3 consumer threads per topic (one per partition)
poll-timeout: 3000
```

9.2 KafkaTemplate – Producer

KafkaTemplate producer – async and targeted sending

[illegible]

```
}

```

9.3 @KafkaListener – Consumer

@KafkaListener – basic, with metadata/manual ack, and batch

[illegible]

```
ack.acknowledge();  
}  
}
```

9.4 Error Handling & Dead Letter Queue

Dead Letter Queue with DefaultErrorHandler and exponential backoff

```
@Configuration  
public class KafkaErrorConfig {  
  
    @Bean  
    public DefaultErrorHandler errorHandler(  
        KafkaTemplate<String, Object> template) {  
  
        // DLQ publisher: failed messages go to topic.DLT  
        DeadLetterPublishingRecoverer recoverer =  
            new DeadLetterPublishingRecoverer(template,  
                (rec, ex) -> new TopicPartition(rec.topic() + ".DLT",  
                    rec.partition()));  
  
        // Exponential backoff: 1s, 2s, 4s, 8s, then DLQ  
        ExponentialBackOffWithMaxRetries backOff =  
            new ExponentialBackOffWithMaxRetries(4);  
        backOff.setInitialInterval(1_000L);  
        backOff.setMultiplier(2.0);  
        backOff.setMaxInterval(16_000L);  
  
        DefaultErrorHandler handler =  
            new DefaultErrorHandler(recoverer, backOff);  
  
        // These exceptions are NOT retried - sent straight to DLQ  
        handler.addNotRetryableExceptions(  
            ValidationException.class,  
            DeserializationException.class  
        );  
        return handler;  
    }  
}  
  
// DLQ consumer - inspect and handle failed messages  
@Service  
public class DlqConsumer {  
  
    @KafkaListener(topics = "order-events.DLT")  
    public void handleDlq(  
        @Payload byte[] rawMessage,  
        @Header(KafkaHeaders.DLT_EXCEPTION_MESSAGE) String errorMsg,  
        @Header(KafkaHeaders.DLT_ORIGINAL_TOPIC) String originalTopic,  
        @Header(KafkaHeaders.DLT_EXCEPTION_STACKTRACE) String stackTrace) {  
        log.error("DLQ: topic={} error={}", originalTopic, errorMsg);  
        alertService.notifyOnDlq(originalTopic, errorMsg);  
    }  
}
```



```
}  
}
```

9.5 Testing with @EmbeddedKafka

@EmbeddedKafka integration test

```
@SpringBootTest  
@EmbeddedKafka(  
    partitions = 1,  
    brokerProperties = {  
        "listeners=PLAINTEXT://localhost:9092",  
        "port=9092"  
    },  
    topics = {"order-events", "order-events.DLT"}  
)  
class OrderEventIntegrationTest {  
  
    @Autowired  
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;  
  
    @Autowired  
    private OrderEventConsumer consumer;  
  
    private final CountDownLatch latch = new CountDownLatch(1);  
    private OrderEvent receivedEvent;  
  
    @Test  
    void produceAndConsumeOrderEvent() throws InterruptedException {  
        OrderEvent event = new OrderEvent("ORDER-1", "CUST-1", 99.99, "PLACED");  
  
        kafkaTemplate.send("order-events", event.getOrderId(), event);  
  
        // Wait up to 10s for consumer to process the message  
        assertThat(latch.await(10, TimeUnit.SECONDS)).isTrue();  
        assertThat(receivedEvent).isNotNull();  
        assertThat(receivedEvent.getOrderId()).isEqualTo("ORDER-1");  
    }  
}
```

CHAPTER 10

Performance Tuning & Best Practices

Producer/Consumer/Broker Tuning, Topic Design & Common Pitfalls

10.1 Performance Tuning Reference

Producer Tuning:

Setting	Recommended Value	Effect
linger.ms	5-20ms	Batch more records; higher latency but higher throughput
batch.size	64-512KB	Larger batches = more efficient I/O
compression.type	snappy/lz4	Reduces network and disk I/O; snappy = low CPU, good ratio
acks	all	Maximum durability; use with idempotence
buffer.memory	64-256MB	More buffer = survive bursts without blocking
max.in.flight.requests	5	5 for high throughput; 1 for strict ordering without idempotence

Consumer Tuning:

Setting	Recommended Value	Effect
fetch.min.bytes	1KB-1MB	Consumer waits for this much data before returning
fetch.max.wait.ms	500ms	Max time to wait for fetch.min.bytes to accumulate
max.poll.records	100-1000	Records per poll(); tune based on processing time
max.poll.interval.ms	300000ms	Must process all records within this time or be kicked
session.timeout.ms	30000ms	Missed heartbeats within this time = consumer dead
heartbeat.interval.ms	3000ms	Should be 1/3 of session.timeout.ms

Broker Tuning:

Setting	Recommended Value	Effect
num.network.threads	8	Threads for network I/O; scale with core count
num.io.threads	16	Threads for disk I/O
log.flush.interval.ms	Long	OS handles fsync; avoid frequent flushes

Setting	Recommended Value	Effect
num.replica.fetchers	4	Threads per broker for replication
log.segment.bytes	1GB	Segment size; smaller = faster deletion, more overhead
log.retention.hours	168 (7d)	How long to keep messages

10.2 Topic Design Best Practices

- **Partition Count** – Start with 3-6x the expected peak consumer count. Hard to reduce later.
- **Replication Factor** – Always 3 in production; never 1 (no fault tolerance) or 2 (split-brain risk).
- **min.insync.replicas** – Set to replication_factor - 1 (e.g., 2 for RF=3). Prevents data loss.
- **Key Design** – Choose keys that distribute load evenly. Skewed keys = hot partitions.
- **Topic Naming** – Use a consistent convention: domain.entity.event (e.g. orders.placed, payments.processed).
- **Retention** – Use time-based (retention.ms) for event streams; use compaction for state topics.
- **Avoid too many topics** – Each partition has overhead (file handles, replication threads).
- **Separate concerns** – One topic per event type; don't mix unrelated events in one topic.

10.3 Common Pitfalls & Troubleshooting

Problem	Symptom	Solution
Consumer Lag Growing	Consumers can't keep up with producers	Increase consumer instances (up to partition count), tune max.poll.records
Rebalancing Storms	Consumers constantly rebalancing	Increase session.timeout.ms, tune max.poll.interval.ms, use static membership
Hot Partitions	One partition gets all traffic	Choose better partition key; use round-robin if key not needed
Message Loss	Messages disappear	Set acks=all, enable.idempotence=true, min.insync.replicas=2
Duplicate Messages	Messages processed more than once	Make consumers idempotent; use Kafka transactions
Slow Consumers	Processing time > max.poll.interval.ms	Reduce max.poll.records, async processing, increase max.poll.interval.ms
Broker Disk Full	Producers start failing	Reduce retention, expand disk, add brokers
OOM in Producer	BufferExhaustedException	Increase buffer.memory, reduce linger.ms, check for slow consumers

+ Monitor these key metrics: consumer lag per group+topic (alert at >10K), under-replicated partitions (alert at >0 for >5min), request rate per broker, disk utilisation (<80%), and producer error rate. Use Confluent Control Center, Kafka UI, or Grafana with the JMX exporter.

! Never decrease the number of partitions on an existing topic – Kafka does not support it without deleting and recreating the topic. Plan your partition count carefully upfront.

Quick Reference – Kafka Properties & CLI Commands

Category	Key Property / Command	Purpose
Producer	acks=all	All ISR replicas acknowledge write
Producer	enable.idempotence=true	Exactly-once per partition
Producer	linger.ms=20	Batch records for up to 20ms
Producer	compression.type=snappy	Compress batches before sending
Producer	transactional.id=app-txn-1	Enable cross-partition transactions
Consumer	group.id=my-group	Consumer group identifier
Consumer	auto.offset.reset=earliest	Start from beginning if no offset
Consumer	enable.auto.commit=false	Manual offset commit for reliability
Consumer	max.poll.records=500	Records per poll() call
Consumer	isolation.level=read_committed	Only read committed transactional msgs
Topic CLI	kafka-topics.sh --create	Create a topic with partitions/replicas
Topic CLI	kafka-topics.sh --describe	Show topic partition/leader/ISR info
Topic CLI	kafka-topics.sh --alter --partitions	Increase partition count
Group CLI	kafka-consumer-groups.sh --describe	Show consumer group lag per partition
Group CLI	kafka-consumer-groups.sh --reset-offsets	Reset group to earliest/latest/offset
Config CLI	kafka-configs.sh --alter	Update broker/topic/user config at runtime
Streams	KStream.filter()	Stateless: filter records by predicate
Streams	KStream.groupByKey().count()	Stateful: count per key
Streams	TimeWindows.ofSizeWithNoGrace()	Tumbling window for time-based aggregation
Spring	@KafkaListener(topics, groupId)	Consume messages from a topic
Spring	kafkaTemplate.send(topic, key, value)	Produce a message
Spring	DeadLetterPublishingRecoverer	Route failed messages to .DLT topic
Spring	@EmbeddedKafka	In-memory Kafka for integration tests